



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 11: Principios SOLID (parte 4)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

10 de Septiembre, 2026

Open-Closed Principle: Modelando las habilidades activas

Open-Closed Principle: Habilidades

Pensemos en las habilidades activas del proyecto:

Open-Closed Principle: Habilidades

Pensemos en las habilidades activas del proyecto:

- **Fireball**: Inflige daño de tipo  **Fire** a todos los enemigos.
- **Wildfire**: Inflige daño de tipo  **Fire** a un enemigo.

Open-Closed Principle: Habilidades

Pensemos en las habilidades activas del proyecto:

- **Fireball**: Inflige daño de tipo  **Fire** a todos los enemigos.
- **Wildfire**: Inflige daño de tipo  **Fire** a un enemigo.

¿Cuál es la diferencia entre estas dos habilidades?

Open-Closed Principle: Habilidades

Pensemos en las habilidades activas del proyecto:

- **Fireball**: Inflige daño de tipo  **Fire** a todos los enemigos.
- **Wildfire**: Inflige daño de tipo  **Fire** a un enemigo.

¿Cuál es la diferencia entre estas dos habilidades?

Ambas habilidades hacen lo mismo (daño de tipo  **Fire**), pero se lo hacen a personas distintas (“a todos los enemigos” y “a un enemigo”)

Open-Closed Principle: Habilidades

Ahora pensemos en estas habilidades

- **Leghold Trap**: Selecciona un enemigo para que, por 2 rondas, este actúe al final de la cola de turnos.
- **Wildfire**: Inflige daño de tipo  **Fire** a un enemigo.

Open-Closed Principle: Habilidades

Ahora pensemos en estas habilidades

- **Leghold Trap**: Selecciona un enemigo para que, por 2 rondas, este actúe al final de la cola de turnos.
- **Wildfire**: Inflige daño de tipo  **Fire** a un enemigo.

¿Cuál es la diferencia entre estas dos habilidades?

Open-Closed Principle: Habilidades

Ahora pensemos en estas habilidades

- **Leghold Trap**: Selecciona un enemigo para que, por 2 rondas, este actúe al final de la cola de turnos.
- **Wildfire**: Inflige daño de tipo  **Fire** a un enemigo.

¿Cuál es la diferencia entre estas dos habilidades?

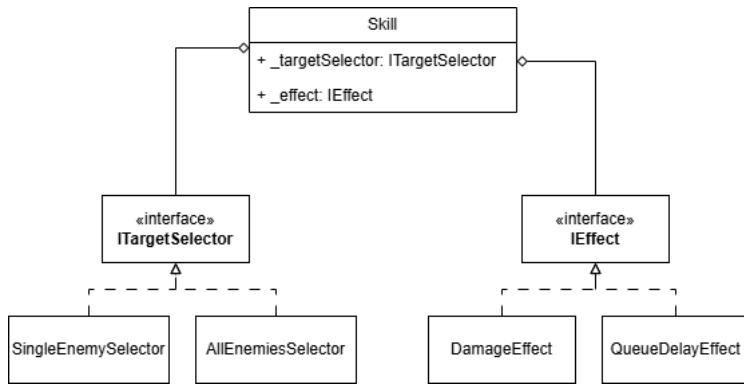
Ambas habilidades hacen cosas distintas (“daño de tipo  **Fire**” y “por 2 rondas, este actúe al final de la cola de turnos”), pero se lo hacen a la misma persona (“a un enemigo”)

Open-Closed Principle: Habilidades

Esto nos da un indicio de como modelar nuestras habilidades: las habilidades crecen en dos direcciones independientes, selección de objetivo y efecto

Open-Closed Principle: Habilidades

Esto nos da un indicio de como modelar nuestras habilidades: las habilidades crecen en dos direcciones independientes, selección de objetivo y efecto



Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?
- ¿Las habilidades, deberían estar en el Modelo, Vista o Controlador?

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?
- ¿Las habilidades, deberían estar en el Modelo, Vista o Controlador?

Si lo pensamos bien:

- Los efectos son parte del Modelo, son datos del juego

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?
- ¿Las habilidades, deberían estar en el Modelo, Vista o Controlador?

Si lo pensamos bien:

- Los efectos son parte del Modelo, son datos del juego
- Los selectores de objetivo, a veces, necesitan llamar a la vista, por lo que no pueden estar en el Modelo

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?
- ¿Las habilidades, deberían estar en el Modelo, Vista o Controlador?

Si lo pensamos bien:

- Los efectos son parte del Modelo, son datos del juego
- Los selectores de objetivo, a veces, necesitan llamar a la vista, por lo que no pueden estar en el Modelo
- Si la habilidad se compone de un `ITargetSelector` y un `IEffect`, no puede estar en el Modelo

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?
- ¿Las habilidades, deberían estar en el Modelo, Vista o Controlador?

Si lo pensamos bien:

- Los efectos son parte del Modelo, son datos del juego
- Los selectores de objetivo, a veces, necesitan llamar a la vista, por lo que no pueden estar en el Modelo
- Si la habilidad se compone de un `ITargetSelector` y un `IEffect`, no puede estar en el Modelo
- Si la habilidad no puede estar en el modelo, no podemos crearlas en el Setup del programa

Open-Closed Principle: Habilidades

Esto tiene algunos problemas problema:

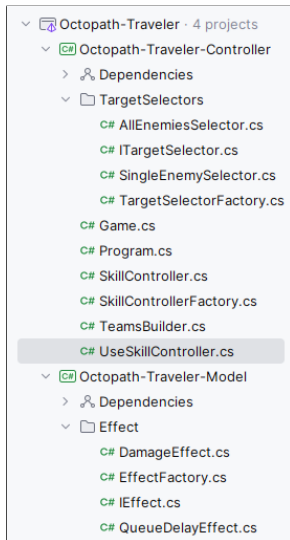
- ¿Los efectos, deberían estar en el Modelo, Vista o Controlador?
- ¿Los selectores de objetivo, deberían estar en el Modelo, Vista o Controlador?
- ¿Las habilidades, deberían estar en el Modelo, Vista o Controlador?

Si lo pensamos bien:

- Los efectos son parte del Modelo, son datos del juego
- Los selectores de objetivo, a veces, necesitan llamar a la vista, por lo que no pueden estar en el Modelo
- Si la habilidad se compone de un `ITargetSelector` y un `IEffect`, no puede estar en el Modelo
- Si la habilidad no puede estar en el modelo, no podemos crearlas en el Setup del programa

Veamos esto a nivel de código

Open-Closed Principle: Habilidades



Open-Closed Principle: Habilidades

Los efectos son una interfaz con un método para aplicar el efecto:

```
1 public interface IEffect
2 {
3     void Apply(Type type, Unit owner, List<Unit> targets, int modifier);
4 }
```

Open-Closed Principle: Habilidades

Los efectos son una interfaz con un método para aplicar el efecto:

```
1 public interface IEffect
2 {
3     void Apply(Type type, Unit owner, List<Unit> targets, int modifier);
4 }
```

Luego cada efecto debe implementar el método `Apply(...)`:

```
1 public class DamageEffect : IEffect
2 {
3     public void Apply(Type type, Unit owner, List<Unit> targets, int modifier)
4     {
5         foreach (Unit target in targets)
6             DamageTarget(type, owner, target, modifier);
7     }
8     private void DamageTarget(Type type, Unit owner, Unit target, int modifier) { /* Effect */ }
9 }
```

```
1 public class QueueDelayEffect : IEffect
2 {
3     public void Apply(Type type, Unit owner, List<Unit> targets, int modifier)
4     {
5         foreach (Unit target in targets)
6             DelayTarget(target);
7     }
8     private void DelayTarget(Unit target) { /* Effect */ }
9 }
```

Open-Closed Principle: Habilidades

Luego, tendremos una factory de efectos

```
1 public static class EffectFactory
2 {
3     public static IEffect Create(string name)
4     {
5         if (name == "Wildfire" || name == "Fireball")
6             return new DamageEffect();
7         if (name == "Leghold Trap")
8             return new QueueDelayEffect();
9         throw new ArgumentException($"Unknown target name: {name}");
10    }
11 }
```

Open-Closed Principle: Habilidades

Los selectores de objetivo son una interfaz con un método para seleccionar al objetivo:

```
1 public interface ITargetSelector
2 {
3     List<Unit> Select(Unit owner, IView view, Team travelers, Team beasts);
4 }
```

Open-Closed Principle: Habilidades

Los selectores de objetivo son una interfaz con un método para seleccionar al objetivo:

```
1 public interface ITargetSelector
2 {
3     List<Unit> Select(Unit owner, IView view, Team travelers, Team beasts);
4 }
```

Luego cada selector debe implementar el método `Select(...)`:

```
1 public class SingleEnemySelector : ITargetSelector
2 {
3     public List<Unit> Select(Unit owner, IView view, Team travelers, Team beasts)
4     {
5         List<Unit> possibleTargets = GetPossibleOptions(beasts);
6         Unit selection = view.AskUserForTarget(possibleTargets);
7         return new List<Unit>() { selection };
8     }
9
10    private List<Unit> GetPossibleOptions(Team beasts){/* Codigo */}
11 }
```

```
1 public class AllEnemiesSelector : ITargetSelector
2 {
3     public List<Unit> Select(Unit owner, IView view, Team travelers, Team beasts)
4     => return GetAliveUnits(beasts);
5
6     private List<Unit> GetAliveUnits(Team beasts){/* Codigo */}
7 }
```

Open-Closed Principle: Habilidades

Luego, tendremos una factory de selectores de objetivo

```
1 public static class TargetSelectorFactory
2 {
3     public static ITargetSelector Create(string name)
4     {
5         if (name == "Fireball")
6             return new AllEnemiesSelector();
7         if (name == "Wildfire" || name == "Leghold Trap")
8             return new SingleEnemySelector();
9         throw new ArgumentException($"Unknown target name: {name}");
10    }
11 }
```

Open-Closed Principle: Habilidades

El controlador de la habilidad es un efecto y un selector de objetivo

```
1 public class SkillController(ITargetSelector selector, IEffect effect)
2 {
3     public void Handle(Unit owner, IView view, Team travelers, Team beasts, Type type, int modifier)
4     {
5         List<Unit> targets = selector.Select(owner, view, travelers, beasts);
6         effect.Apply(type, owner, targets, modifier);
7     }
8 }
```

También tendremos una factory para estos:

```
1 public static class SkillControllerFactory
2 {
3     public static SkillController Create(string name)
4     {
5         ITargetSelector selector = TargetSelectorFactory.Create(name);
6         IEffect effect = EffectFactory.Create(name);
7         return new SkillController(selector, effect);
8     }
9 }
```

Open-Closed Principle: Habilidades

Crearemos un controlador de la habilidad cuando el usuario seleccione que habilidad quiere utilizar, no los pre-cargaremos en el modelo (no podremos aunque queramos).

```
1 public class UseSkillController(IView view)
2 {
3     public void Handle()
4     {
5         ActiveSkill skill = view.AskUserForSkill(GetUsableSkills());
6         SkillController skillController = SkillControllerFactory.
Create(skill.Name);
7         // Code
8         skillController.Handle(owner, view, travelers, beasts, type,
modifier);
9     }
10
11     private List<ActiveSkill> GetUsableSkills(){/* Codigo */}
12 }
```

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

- Cada efecto necesita distintos parámetros para funcionar

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

- Cada efecto necesita distintos parámetros para funcionar
- Si les pasamos estos parámetros en el método `Apply(...)`, entonces cambiaremos `IEffect` cada vez que agreguemos una habilidad distinta

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

- Cada efecto necesita distintos parámetros para funcionar
- Si les pasamos estos parámetros en el método `Apply(...)`, entonces cambiaremos `IEffect` cada vez que agreguemos una habilidad distinta
- Esto rompe el Open-Closed Principle ¿Por qué?

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

- Cada efecto necesita distintos parámetros para funcionar
- Si les pasamos estos parámetros en el método `Apply(...)`, entonces cambiaremos `IEffect` cada vez que agreguemos una habilidad distinta
- Esto rompe el Open-Closed Principle ¿Por qué?

La modelación de los selectores de objetivos tiene problemas:

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

- Cada efecto necesita distintos parámetros para funcionar
- Si les pasamos estos parámetros en el método `Apply(...)`, entonces cambiaremos `IEffect` cada vez que agreguemos una habilidad distinta
- Esto rompe el Open-Closed Principle ¿Por qué?

La modelación de los selectores de objetivos tiene problemas:

- Cada efecto necesita distintos parámetros y estos se reciben en el método `Select(...)`

Open-Closed Principle: Habilidades

Esta modelación de los efectos tiene grandes problemas:

- Cada efecto necesita distintos parámetros para funcionar
- Si les pasamos estos parámetros en el método `Apply(...)`, entonces cambiaremos `IEffect` cada vez que agreguemos una habilidad distinta
- Esto rompe el Open-Closed Principle ¿Por qué?

La modelación de los selectores de objetivos tiene problemas:

- Cada efecto necesita distintos parámetros y estos se reciben en el método `Select(...)`
- El método `Select(...)` siempre retorna una `List<Unit>`. Esto hará difícil agregar selecciones de elementos que no sean unidades (como armas).

Open-Closed Principle: Habilidades

Para solucionar esto, podemos aprovechar nuestro `GameState`.

Open-Closed Principle: Habilidades

Para solucionar esto, podemos aprovechar nuestro `GameState`. Esta clase puede tener parámetros globales, como los equipos, la unidad que esta actuando y la lista de objetivos

Open-Closed Principle: Habilidades

Para solucionar esto, podemos aprovechar nuestro GameState. Esta clase puede tener parámetros globales, como los equipos, la unidad que esta actuando y la lista de objetivos

```
1 public class GameState
2 {
3     public Team TravelersTeam { get; set; }
4     public Team BeastsTeam { get; set; }
5     public Unit ActingUnit { get; set; }
6     public List<Unit> Targets { get; set; } = new();
7     // Más parametros
8 }
```

Open-Closed Principle: Habilidades

Los selectores de objetivo ya no retornar unidades y no recibe nada:

```
1 public interface ITargetSelector
2 {
3     void Select();
4 }
```

Open-Closed Principle: Habilidades

Los selectores de objetivo ya no retornar unidades y no recibe nada:

```
1 public interface ITargetSelector
2 {
3     void Select();
4 }
```

Los selectores reciben el GameState en el constructor y le agregan unidades:

```
1 public class AllEnemiesSelector(GameState state) : ITargetSelector
2 {
3     public void Select()
4     {
5         List<Unit> units = GetAliveUnits();
6         state.Targets.AddRange(units);
7     }
8     private List<Unit> GetAliveUnits() { /* Usa state.BeastTeam */ }
9 }
```

```
1 public class SingleEnemySelector(GameState state, IView view) : ITargetSelector
2 {
3     public void Select()
4     {
5         List<Unit> possibleTargets = GetPossibleOptions();
6         Unit selection = view.AskUserForTarget(possibleTargets);
7         state.Targets.Add(selection);
8     }
9     private List<Unit> GetPossibleOptions() { /* Usa state.BeastTeam */ }
10 }
```

Open-Closed Principle: Habilidades

Los selectores de objetivo pueden recibir argumentos distintos desde el constructor.

Open-Closed Principle: Habilidades

Los selectores de objetivo pueden recibir argumentos distintos desde el constructor. Esto se maneja en la factory

```
1 public static class TargetSelectorFactory
2 {
3     public static ITargetSelector Create(ActiveSkill skill, GameState
state, IView view)
4     {
5         if (skill.Name == "Fireball")
6             return new AllEnemiesSelector(state);
7         if (skill.Name == "Wildfire" || skill.Name == "Leghold Trap")
8             return new SingleEnemySelector(state, view);
9         throw new ArgumentException($"Unknown target name: {skill.Name
}");
10    }
11 }
```

Open-Closed Principle: Habilidades

Al igual que los selectores, los efectos ya no reciben nada en la interfaz:

```
1 public interface IEffect
2 {
3     void Apply();
4 }
```

Open-Closed Principle: Habilidades

Al igual que los selectores, los efectos ya no reciben nada en la interfaz:

```
1 public interface IEffect
2 {
3     void Apply();
4 }
```

Luego, estos recorreran la lista de objetivos de GameState

```
1 public abstract class BaseEffect(GameState state) : IEffect
2 {
3     public void Apply()
4     {
5         foreach (Unit target in state.Targets)
6             ApplyEffectTo(target);
7     }
8     protected abstract void ApplyEffectTo(Unit target);
9 }
```

```
1 public class DamageEffect(GameState state, int modifier, Type type) : BaseEffect(state)
2 {
3     protected override void ApplyEffectTo(Unit target) { /* Usa modifier y type */ }
4 }
```

```
1 public class QueueDelayEffect(GameState state) : BaseEffect(state)
2 {
3     protected override void ApplyEffectTo(Unit target) { /*Delay effect*/ }
4 }
```

Open-Closed Principle: Habilidades

Al igual que los selectores de objetivo, los efectos también pueden recibir cosas distintas en sus constructores

```
1 public static class EffectFactory
2 {
3     public static IEffect Create(ActiveSkill skill, GameState state)
4     {
5         if (skill.Name == "Wildfire" || skill.Name == "Fireball")
6             return new DamageEffect(state, skill.Modifier, skill.Type)
7         ;
8         if (skill.Name == "Leghold Trap")
9             return new QueueDelayEffect(state);
10        throw new ArgumentException($"Unknown target name: {skill.
11        Name}");
12    }
13 }
```

Open-Closed Principle: Habilidades

El controlador de la habilidades tampoco necesitará recibir argumentos

```
1 public class SkillController(ITargetSelector selector, IEffect effect)
2 {
3     public void Handle()
4     {
5         selector.Select();
6         effect.Apply();
7     }
8 }
```

Open-Closed Principle: Habilidades

El controlador de la habilidades tampoco necesitará recibir argumentos

```
1 public class SkillController(ITargetSelector selector, IEffect effect)
2 {
3     public void Handle()
4     {
5         selector.Select();
6         effect.Apply();
7     }
8 }
```

La fabrica seguirá siendo simple:

```
1 public static class SkillControllerFactory
2 {
3     public static SkillController Create(ActiveSkill skill, GameState state, IView view)
4     {
5         ITargetSelector selector = TargetSelectorFactory.Create(skill, state, view);
6         IEffect effect = EffectFactory.Create(skill, state);
7         return new SkillController(selector, effect);
8     }
9 }
```

Open-Closed Principle: Habilidades

Al momento de usar las habilidades, ya no necesitaremos llevar registro de tantos argumentos

```
1 public class UseSkillController(GameState state, IView view)
2 {
3     public void Handle()
4     {
5         ActiveSkill skill = view.AskUserForSkill(GetUsableSkills());
6         SkillController skillController = SkillControllerFactory.
7         Create(skill, state, view);
8         // Code
9         skillController.Handle();
10    }
11    private List<ActiveSkill> GetUsableSkills(){/*Código*/}
```

Open-Closed Principle: Habilidades

Al momento de usar las habilidades, ya no necesitaremos llevar registro de tantos argumentos

```
1 public class UseSkillController(GameState state, IView view)
2 {
3     public void Handle()
4     {
5         ActiveSkill skill = view.AskUserForSkill(GetUsableSkills());
6         SkillController skillController = SkillControllerFactory.
7         Create(skill, state, view);
8         // Code
9         skillController.Handle();
10    }
11    private List<ActiveSkill> GetUsableSkills(){/*Código*/}
```

Terminamos con un código que respeta el OCP y no nos amarra

Open-Closed Principle: Mostrando resultados

Open-Closed Principle: Resultados

Luego de ejecutar una habilidad, tenemos que mostrarle al usuario el resultado de la ejecución. Para esto, es normal tener un método en la vista:

```
1 public interface IView
2 {
3     void ShowInvalidTeamMessage();
4     TeamsInfo SelectTeamInfo();
5     Unit AskUserForTarget(List<Unit> possibleUnits);
6     ActiveSkill AskUserForSkill(List<ActiveSkill> skills);
7     void AnnounceFireDamage();
8 }
```

Open-Closed Principle: Resultados

Cuando agreguemos más habilidades, posiblemente le agregaremos más métodos a la interfaz

```
1 public interface IView
2 {
3     void ShowInvalidTeamMessage();
4     TeamsInfo SelectTeamInfo();
5     Unit AskUserForTarget(List<Unit> possibleUnits);
6     ActiveSkill AskUserForSkill(List<ActiveSkill> skills);
7     void AnnounceFireDamage();
8     void AnnounceQueueDelay();
9 }
```

Open-Closed Principle: Resultados

Cuando agreguemos más habilidades, posiblemente le agregaremos más métodos a la interfaz

```
1 public interface IView
2 {
3     void ShowInvalidTeamMessage();
4     TeamsInfo SelectTeamInfo();
5     Unit AskUserForTarget(List<Unit> possibleUnits);
6     ActiveSkill AskUserForSkill(List<ActiveSkill> skills);
7     void AnnounceFireDamage();
8     void AnnounceQueueDelay();
9 }
```

Esto rompe el OCP ¿Por qué?

Open-Closed Principle: Resultados

Si seguimos en este camino, modificaremos la interfaz de la vista cada vez que agreguemos un nuevo efecto

```
1 public interface IView
2 {
3     void ShowInvalidTeamMessage();
4     TeamsInfo SelectTeamInfo();
5     Unit AskUserForTarget(List<Unit> possibleUnits);
6     ActiveSkill AskUserForSkill(List<ActiveSkill> skills);
7     void AnnounceFireDamage();
8     void AnnounceQueueDelay();
9     void AnnounceQueuePriority();
10    void AnnounceIceDamage();
11    void AnnounceSpearDamage();
12    void AnnounceSwordDamage();
13    void AnnounceStaveDamage();
14    /* Más métodos */
15 }
```

Open-Closed Principle: Resultados

Los problemas principales:

Open-Closed Principle: Resultados

Los problemas principales:

- A medida que agreguemos nuevas habilidades, tendremos que agregar nuevos tipos de mensajes para anunciar los resultados

Open-Closed Principle: Resultados

Los problemas principales:

- A medida que agreguemos nuevas habilidades, tendremos que agregar nuevos tipos de mensajes para anunciar los resultados
- Si agregamos un método por tipo de mensaje de resultado, romperemos el Open-Closed Principle

Open-Closed Principle: Resultados

Los problemas principales:

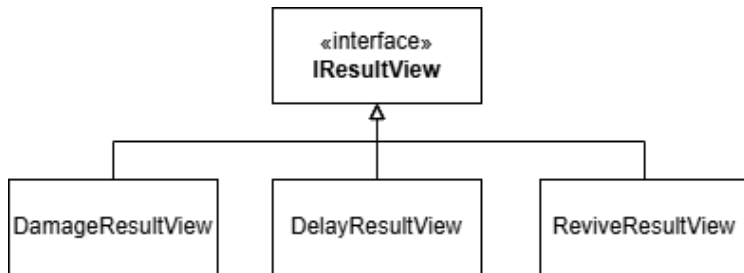
- A medida que agreguemos nuevas habilidades, tendremos que agregar nuevos tipos de mensajes para anunciar los resultados
- Si agregamos un método por tipo de mensaje de resultado, romperemos el Open-Closed Principle
- Tenemos que encontrar una manera de agregar nuevos tipos de mensaje de resultado a cada entrega. ¿Cómo?

Open-Closed Principle: Resultados

Los problemas principales:

- A medida que agreguemos nuevas habilidades, tendremos que agregar nuevos tipos de mensajes para anunciar los resultados
- Si agregamos un método por tipo de mensaje de resultado, romperemos el Open-Closed Principle
- Tenemos que encontrar una manera de agregar nuevos tipos de mensaje de resultado a cada entrega. ¿Cómo?

Solución: Polimorfismo



Open-Closed Principle: Resultados

Para que esto funcione, primero tenemos que encontrar una forma de entregarle a la vista información sobre el resultado de ejecutar una acción.

Open-Closed Principle: Resultados

Para que esto funcione, primero tenemos que encontrar una forma de entregarle a la vista información sobre el resultado de ejecutar una acción. Para esto podemos usar una EDD

```
1 public class ActionResult
2 {
3     public Unit Source { get; set; }
4     public Unit Target { get; set; }
5     public ActionType Type { get; set; }
6     public int Damage { get; set; }
7     /* Más parametros */
8 }
```

Open-Closed Principle: Resultados

De forma similar a los objetivos, podemos llevar registros de los resultados de las acciones en la clase GameState

```
1 public class GameState
2 {
3     public Team TravelersTeam { get; set; }
4     public Team BeastsTeam { get; set; }
5     public Unit ActingUnit { get; set; }
6     public List<Unit> Targets { get; set; } = new();
7     public List<ActionResult> Results { get; set; } = new();
8     // Más parametros
9 }
```

Luego, cada efecto deberá rellenar la lista de resultados

Open-Closed Principle: Resultados

```
1 public abstract class BaseEffect(GameState state) : IEffect
2 {
3     public void Apply()
4     {
5         foreach (Unit target in state.Targets)
6         {
7             ApplyEffectTo(target);
8             RegisterActionResult(target);
9         }
10    }
11    protected abstract void ApplyEffectTo(Unit target);
12    protected abstract void RegisterActionResult(Unit target);
13 }
```

Open-Closed Principle: Resultados

```
1 public abstract class BaseEffect(GameState state) : IEfect
2 {
3     public void Apply()
4     {
5         foreach (Unit target in state.Targets)
6         {
7             ApplyEffectTo(target);
8             RegisterActionResult(target);
9         }
10    }
11    protected abstract void ApplyEffectTo(Unit target);
12    protected abstract void RegisterActionResult(Unit target);
13 }
```

```
1 public class DamageEffect(GameState state, int modifier, Type type) : BaseEffect(state)
2 {
3     protected override void ApplyEffectTo(Unit target) { /* Usa modifier y type */ }
4     protected override void RegisterActionResult(Unit target)
5     {
6         ActionResult result = new ActionResult();
7         result.Source = state.ActingUnit;
8         result.Target = target;
9         /* Rellenar parametros */
10        state.Results.Add(result);
11    }
12 }
```

Open-Closed Principle: Resultados

La interfaz de la vista debería tener un método muy simple para mostrar los resultados de las acciones

```
1 public interface IView
2 {
3     void ShowInvalidTeamMessage();
4     TeamsInfo SelectTeamInfo();
5     Unit AskUserForTarget(List<Unit> possibleUnits);
6     ActiveSkill AskUserForSkill(List<ActiveSkill> skills);
7     void AnnounceActionResult();
8     /* Más métodos */
9 }
```

Open-Closed Principle: Resultados

La interfaz de la vista debería tener un método muy simple para mostrar los resultados de las acciones

```
1 public interface IView
2 {
3     void ShowInvalidTeamMessage();
4     TeamsInfo SelectTeamInfo();
5     Unit AskUserForTarget(List<Unit> possibleUnits);
6     ActiveSkill AskUserForSkill(List<ActiveSkill> skills);
7     void AnnounceActionResult();
8     /* Más métodos */
9 }
```

Luego, debemos implementar esto en la vista de consola

Open-Closed Principle: Resultados

La vista de consola tendrá como atributo al GameState. Luego, se utilizará la lista de ActionResult que se encuentra en GameState para armar un objeto de tipo IActionResultView

```
1 public class MainConsoleView : IView
2 {
3     private View _view;
4     private string _teamsFolder;
5     private GameState _gameState;
6
7     /* Más métodos */
8
9     public void AnnounceActionResult()
10    {
11        IActionResultView resultView = ResultViewFactory.Create(_gameState);
12        resultView.Announce();
13    }
14 }
```

Open-Closed Principle: Resultados

La vista de consola tendrá como atributo al GameState. Luego, se utilizará la lista de ActionResult que se encuentra en GameState para armar un objeto de tipo IActionResultView

```
1 public class MainConsoleView : IView
2 {
3     private View _view;
4     private string _teamsFolder;
5     private GameState _gameState;
6
7     /* Más métodos */
8
9     public void AnnounceActionResult()
10    {
11        IActionResultView resultView = ResultViewFactory.Create(_gameState);
12        resultView.Announce();
13    }
14 }
```

La clase ResultViewFactory estará encargada de construir la vista correcta para los objetos ActionResult entregados. Esta clase también se encargará de ordenarlos y agruparlos en caso de ser necesario.

The Open Closed Principle

Habilidades pasivas

The Open Closed Principle: Habilidades pasivas

Las habilidades pasivas funcionan de forma diferente a las habilidades activas:

The Open Closed Principle: Habilidades pasivas

Las habilidades pasivas funcionan de forma diferente a las habilidades activas:

- Las habilidades pasivas no se activan con una acción del usuario. Se activan de forma automática

The Open Closed Principle: Habilidades pasivas

Las habilidades pasivas funcionan de forma diferente a las habilidades activas:

- Las habilidades pasivas no se activan con una acción del usuario. Se activan de forma automática
- Las habilidades pasivas se activan en momentos específicos del juego. No todas se activan al mismo tiempo

The Open Closed Principle: Habilidades pasivas

Las habilidades pasivas funcionan de forma diferente a las habilidades activas:

- Las habilidades pasivas no se activan con una acción del usuario. Se activan de forma automática
- Las habilidades pasivas se activan en momentos específicos del juego. No todas se activan al mismo tiempo

Veamos unos ejemplos:

- **Elemental Augmentation:** Aumenta en 50 el stat ElemAtk de la unidad que porte la habilidad.
- **Patience:** Si luego de jugar todos los turnos de una ronda, el stat HP y el stat SP de la unidad son par, entonces la unidad obtiene un turno adicional antes de finalizar la ronda. Uso único por ronda.
- **Persistence:** Todos los cambios de aumento de stat que la unidad reciba durarán un turno extra.
- **Hang Tough:** Cuando la unidad está sobre el 10% de su HP máximo, cualquier ataque que podría reducir su HP a 0 lo dejará a 1 de HP.

The Open Closed Principle: Habilidades pasivas

Una opción que suena razonable: Reutilizar la estructura de mis habilidades activas

```
1 public interface IEffect
2 {
3     void ApplyActiveEffect();
4     void ApplyAtStartOfGame();
5     void ApplyAtstartOfTurn();
6     void ApplyAtEndOfTurn();
7 }
```

The Open Closed Principle: Habilidades pasivas

Una opción que suena razonable: Reutilizar la estructura de mis habilidades activas

```
1 public interface IEffect
2 {
3     void ApplyActiveEffect();
4     void ApplyAtStartOfGame();
5     void ApplyAtstartOfTurn();
6     void ApplyAtEndOfTurn();
7 }
```

Esto rompe el OCP y el LSP ¿Por qué?

The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

- Todas las actividades pasivas se activan en momentos distintos

The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

- Todas las actividades pasivas se activan en momentos distintos
- Se activan cuando suceden ciertos eventos

The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

- Todas las actividades pasivas se activan en momentos distintos
- Se activan cuando suceden ciertos eventos
- Necesitamos una forma de que los efectos se activen de forma automática cuando sucedan estos eventos

The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

- Todas las actividades pasivas se activan en momentos distintos
- Se activan cuando suceden ciertos eventos
- Necesitamos una forma de que los efectos se activen de forma automática cuando sucedan estos eventos
- No queremos poner condicionales en todo nuestro código cada vez que agregamos una nueva habilidad pasiva

The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

- Todas las actividades pasivas se activan en momentos distintos
- Se activan cuando suceden ciertos eventos
- Necesitamos una forma de que los efectos se activen de forma automática cuando sucedan estos eventos
- No queremos poner condicionales en todo nuestro código cada vez que agregamos una nueva habilidad pasiva
- Necesitamos “notificar” a las habilidades pasivas cuando sucedan los eventos

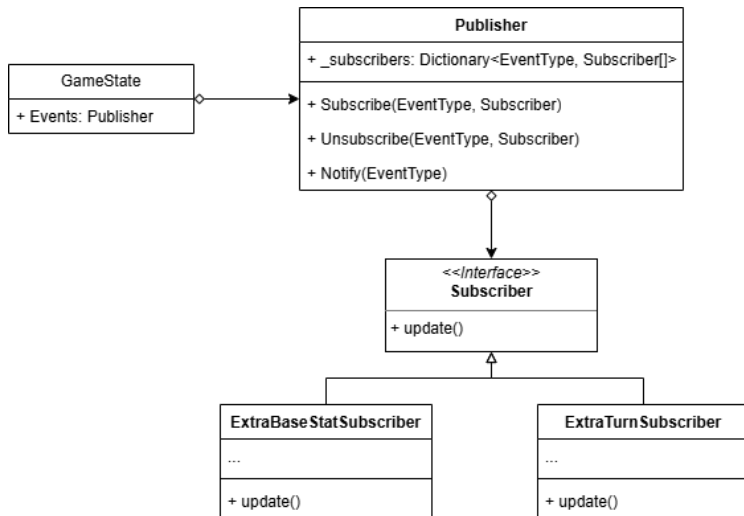
The Open Closed Principle: Habilidades pasivas

Veamos el problema más detenidamente

- Todas las actividades pasivas se activan en momentos distintos
- Se activan cuando suceden ciertos eventos
- Necesitamos una forma de que los efectos se activen de forma automática cuando sucedan estos eventos
- No queremos poner condicionales en todo nuestro código cada vez que agregamos una nueva habilidad pasiva
- Necesitamos “notificar” a las habilidades pasivas cuando sucedan los eventos

Hay un patrón de diseño que soluciona este problema ¿Cuál?

The Open Closed Principle: Habilidades pasivas



The Open Closed Principle: Habilidades pasivas

Lo primero que tenemos que hacer es añadir un nuevo atributo a GameState

```
1 public class GameState
2 {
3     public Team TravelersTeam { get; set; }
4     public Team BeastsTeam { get; set; }
5     public Unit ActingUnit { get; set; }
6     public List<Unit> Targets { get; set; } = new();
7     public List<ActionResult> Results { get; set; } = new();
8     public Publisher Events { get; set; } = new();
9     // Más parametros
10 }
```

The Open Closed Principle: Habilidades pasivas

Lo primero que tenemos que hacer es añadir un nuevo atributo a GameState

```
1 public class GameState
2 {
3     public Team TravelersTeam { get; set; }
4     public Team BeastsTeam { get; set; }
5     public Unit ActingUnit { get; set; }
6     public List<Unit> Targets { get; set; } = new();
7     public List<ActionResult> Results { get; set; } = new();
8     public Publisher Events { get; set; } = new();
9     // Más parametros
10 }
```

Luego, definiremos nuestro Publisher

The Open Closed Principle: Habilidades pasivas

```
1 public class Publisher
2 {
3     private readonly Dictionary<EventType, List<Subscriber>> _subscribers = new();
4
5     public void Subscribe(EventType type, Subscriber subscriber)
6     {
7         if (!_subscribers.ContainsKey(type))
8             _subscribers[type] = new List<Subscriber>();
9         _subscribers[type].Add(subscriber);
10    }
11
12    public void Unsubscribe(EventType type, Subscriber subscriber)
13    {
14        if (_subscribers.ContainsKey(type))
15            _subscribers[type].Remove(subscriber);
16    }
17
18    public void Notify(EventType type)
19    {
20        if (_subscribers.ContainsKey(type))
21            foreach (var handler in _subscribers[type])
22                handler.Update();
23    }
24 }
```

The Open Closed Principle: Habilidades pasivas

Nuestros subscriptores representarán nuestras habilidades pasivas

```
1 public interface Subscriber
2 {
3     void Update();
4 }
```

The Open Closed Principle: Habilidades pasivas

Nuestros subscriptores representarán nuestras habilidades pasivas

```
1 public interface Subscriber
2 {
3     void Update();
4 }
```

Cada subscriptor implementará el efecto de una habilidad pasiva

```
1 public class ExtraBaseStatSubscriber(GameState state) : Subscriber
2 {
3     public void Update(){/* Efecto de la habilidad */}
4 }
```

```
1 public class ExtraTurnSubscriber(GameState state) : Subscriber
2 {
3     public void Update(){/* Efecto de la habilidad */}
4 }
```

The Open Closed Principle: Habilidades pasivas

Para aplicar los efectos, basta con llamar al método `Notify(...)` de el atributo `Events` de la clase `GameState` con el atributo correcto

```
1 public class RoundController(GameState state)
2 {
3     public void Handle()
4     {
5         while(!state.TurnQueue.IsEmpty())
6         {
7             Unit actingUnit = state.TurnQueue.GetNextActingUnit();
8             RunTurn(actingUnit);
9         }
10        state.Events.Notify(EventType.RoundEnded);
11    }
12    /* Otros métodos */
13 }
```